

Sesión de Aprendizaje 4: PHP Avanzado

Duración: 2-3 horas

Nivel: Avanzado

Objetivos:

- Comprender y aplicar las características avanzadas de PHP.
 - Manejar fechas, archivos, cookies y sesiones de manera efectiva.
 - Utilizar filtros y funciones de devolución de llamada.
 - Trabajar con JSON y manejar excepciones.
-

1. Introducción a PHP Avanzado

PHP (Hypertext Preprocessor) es un lenguaje de programación del lado del servidor que se utiliza principalmente para el desarrollo web. En esta sesión, exploraremos características avanzadas que permiten crear aplicaciones más robustas y eficientes.

Importancia de PHP Avanzado:

- Mejora la seguridad.
 - Optimiza el rendimiento.
 - Facilita el manejo de datos y sesiones de usuario.
-

2. Fecha y Hora en PHP

Teoría

PHP ofrece varias funciones para manejar fechas y horas. La función `date()` permite formatear la fecha actual en varios estilos.

Ejemplos

php

Copiar código

- `echo date("Y-m-d H:i:s"); // Muestra la fecha y hora actual`

Manipulación de Fechas

Para realizar operaciones con fechas, PHP tiene funciones como `strtotime()` que convierte una cadena de texto en un timestamp y `time()` que devuelve el timestamp actual.

Ejemplo de Diferencia entre Fechas

php

Copiar código

- `$fecha1 = strtotime("2023-09-01");`
- `$fecha2 = strtotime("2023-09-30");`
- `$diferencia = ($fecha2 - $fecha1) / (60 * 60 * 24); // Días`
- `echo "Diferencia en días: " . $diferencia;`

Ejercicio Práctico

Crea un script que muestre la fecha y hora actual en diferentes formatos y calcule la diferencia entre dos fechas ingresadas por el usuario.

3. Inclusión de Archivos en PHP

Teoría

La inclusión de archivos permite dividir el código en múltiples archivos, facilitando su gestión y reutilización. Existen dos funciones principales: `include` y `require`.

Ejemplos

php

Copiar código

- `// include - si el archivo no se encuentra, continúa la ejecución`
- `include 'archivo.php';`
-
- `// require - si el archivo no se encuentra, detiene la ejecución`
- `require 'archivo.php';`

Diferencias

- `require`: detiene la ejecución si el archivo no se encuentra.
- `include`: emite una advertencia y continúa la ejecución.

Ejercicio Práctico

Crea un archivo de funciones (por ejemplo, `funciones.php`) y utilízalo en otro script. Define al menos tres funciones.

4. Manejo de Archivos en PHP

Teoría

PHP permite interactuar con el sistema de archivos. Puedes abrir, leer, escribir y cerrar archivos utilizando varias funciones.

4.1 Abrir y Leer Archivos

php

Copiar código

- `$archivo = fopen("archivo.txt", "r"); // Abrir en modo lectura`
- `$contenido = fread($archivo, filesize("archivo.txt"));`
- `fclose($archivo);`

4.2 Crear y Escribir Archivos

php

Copiar código

- `$archivo = fopen("nuevo_archivo.txt", "w"); // Abrir en modo escritura`
- `fwrite($archivo, "Contenido de ejemplo");`
- `fclose($archivo);`

4.3 Carga de Archivos

Para manejar archivos subidos por los usuarios, PHP ofrece la función `move_uploaded_file()`.

Ejemplo

php

Copiar código

- `if (move_uploaded_file($_FILES['file']['tmp_name'], 'uploads/'`
- `. $_FILES['file']['name'])) {`
- `echo "Archivo subido correctamente.";`
- `}`

Ejercicio Práctico

Crea un script que permita a los usuarios subir un archivo y lo guarde en el servidor.

5. Cookies en PHP (2 s)

Teoría

Las cookies son pequeñas piezas de datos almacenadas en el navegador del usuario. PHP permite crear, leer y eliminar cookies.

Ejemplos

Crear una Cookie:

php

Copiar código

- `setcookie("usuario", "nombre", time() + (86400 * 30), "/"); //`
`Válido por 30 días`

Leer una Cookie:

php

Copiar código

- `if (isset($_COOKIE["usuario"])) {`
- `echo "Hola " . $_COOKIE["usuario"];`
- `}`

Ejercicio Práctico

Crea un script que gestione cookies para recordar el nombre del usuario entre sesiones.

6. Sesiones en PHP

Teoría

Las sesiones permiten almacenar información en el servidor y asociarla con un usuario específico. Son útiles para mantener el estado entre diferentes s.

Ejemplos

Iniciar una Sesión:

php

Copiar código

- `session_start();`
- `$_SESSION["nombre"] = "valor"; // Guardar información`

Leer y Cerrar Sesiones:

php

Copiar código

- `echo $_SESSION["nombre"];`
- `session_destroy(); // Termina la sesión`

Ejercicio Práctico

Implementa un sistema de inicio y cierre de sesión básico que recuerde al usuario entre s.

7. Filtros en PHP

Teoría

PHP proporciona funciones para validar y sanitizar datos, mejorando la seguridad y la integridad de la aplicación.

Ejemplos

Uso de `filter_var()`:

php

Copiar código

- `$email = filter_var($input, FILTER_VALIDATE_EMAIL);`

Filtros Avanzados

Se pueden usar filtros predefinidos o personalizados para manejar datos más complejos.

Ejercicio Práctico

Crea un formulario que utilice filtros para validar entradas, como correos electrónicos y números.

8. Funciones de Devolución de Llamada en PHP

Teoría

Las funciones de devolución de llamada permiten pasar funciones como argumentos a otras funciones, proporcionando una forma flexible de manipular datos.

Ejemplos

php

Copiar código

- `function miFuncion($valor) {`
- `return $valor * 2;`
- `}`
- `$resultado = array_map("miFuncion", [1, 2, 3]);`

Uso con `usort()`

php

Copiar código

- `usort($array, function($a, $b) {`

- `return $a - $b; // Ordenar de menor a mayor`
- `});`

Ejercicio Práctico

Implementa funciones de devolución de llamada para manipular arreglos de diferentes formas.

9. Trabajando con JSON en PHP

Teoría

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos. PHP ofrece funciones para codificar y decodificar JSON.

Ejemplos

Codificación:

php

Copiar código

- `$data = json_encode($array);`

Decodificación:

php

Copiar código

- `$array = json_decode($data, true); // true para convertir a array asociativo`

Ejercicio Práctico

Crea una API simple que devuelva datos en formato JSON al realizar una solicitud.

10. Manejo de Excepciones en PHP

Teoría

Las excepciones son una forma de manejar errores en PHP. Utilizar `try`, `catch` y `finally` permite gestionar errores de manera más controlada.

Ejemplos

php

Copiar código

- `try {`
- `// Código que puede lanzar una excepción`
- `} catch (Exception $e) {`
- `echo "Error: " . $e->getMessage();`
- `} finally {`
- `// Código que se ejecuta siempre`
- `}`

Excepciones Personalizadas

Puedes crear excepciones personalizadas para casos específicos.

Ejercicio Práctico

Implementa un manejo de excepciones en un script que interactúe con archivos.

11. Buenas Prácticas y Conclusiones

- **Resumen:** Se han cubierto conceptos avanzados de PHP que permiten crear aplicaciones más seguras y eficientes.
- **Consejos:**
 - Comentar y documentar el código.
 - Utilizar herramientas de análisis estático.
 - Seguir estándares de codificación.

12. Recursos Adicionales (1)

- Documentación oficial de PHP: [PHP.net](https://www.php.net)
 - Libros recomendados sobre PHP avanzado.
 - Tutoriales y cursos en línea.
-

Tarea Opcional (1)

Desarrolla una aplicación web sencilla que utilice todos los conceptos aprendidos, como manejo de sesiones, cookies, carga de archivos y manejo de excepciones.